ARRAYS

Student Notes: Definitions, Characteristics, Advantages, Disadvantages, Initialization and Types

Write the shaded **Answer** boxes in your notebook. Copy the diagrams neatly with index numbers.

1. Definition of an Array

Answer: An **array** is a collection of elements of the **same data type** stored in **continuous (adjacent) memory locations** under **one common name**. Each element is accessed using a number called an **index** (subscript), which starts from 0.

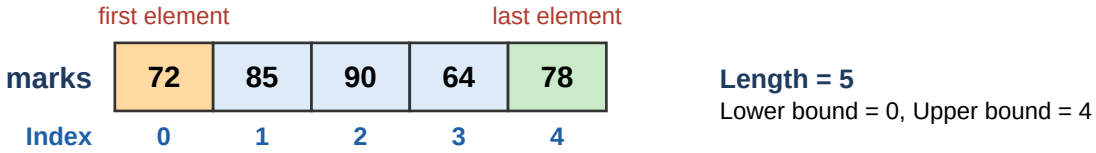

Figure 1: A one-dimensional array named marks with 5 elements

Important terms

Term	Meaning
Element	Each individual value stored in the array (e.g. 85)
Index (subscript)	Position number of an element; the first element has index 0
Length / size	Total number of elements in the array
Lower bound	Smallest valid index = 0
Upper bound	Largest valid index = length - 1
Base address	Memory address of the first element (index 0)
Traversal	Visiting every element of the array once

Why do we need arrays?

To store 100 marks without arrays, we would need 100 different variables. With an array, one name stores all of them and a loop can process them easily.

```
Without array:  m1, m2, m3, ... m100      (100 variables)
With array:     marks[0] ... marks[99]    (1 name, 100 values)
```

2. Characteristics of an Array

1. **Homogeneous:** all elements are of the same data type (in C and Java).
2. **Contiguous memory:** elements are stored in adjacent memory locations.
3. **Indexed:** every element has a unique index, and indexing starts from 0.
4. **Random (direct) access:** any element can be accessed directly using its index in constant time, O(1).
5. **Fixed size:** in a static array, the size is decided at the time of creation and cannot change.
6. **Ordered:** elements remain in the sequence in which they were stored.
7. **Single name:** one array name represents all elements; the name refers to the base address.
8. **Equal element size:** every element occupies the same number of bytes.

Memory representation

Answer: Because elements are placed one after another, the address of any element can be calculated: **Address of a[i] = Base address + (i x size of one element)**

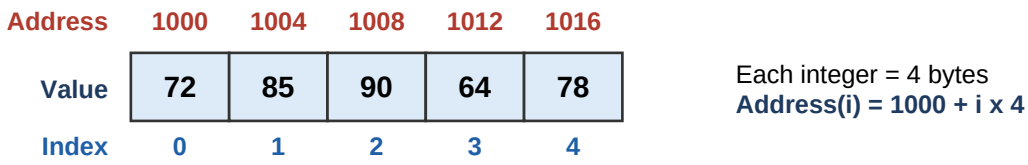

Figure 2: Integer array stored in memory (base address 1000, 4 bytes per integer)

Example: address of index 3 = 1000 + (3 x 4) = **1012**.

3. Declaration and Initialization

Answer: Declaration means telling the compiler the name, data type and size of the array. **Initialization** means assigning values to the array elements, either at the time of declaration or later.

Ways to declare and initialize

Method	C	Java	Python
Declaration only	`int a[5];`	`int[] a = new int[5];`	`a = [0] * 5`
Declaration with values	`int a[5] = {10,20,30,40,50};`	`int[] a = {10,20,30,40,50};`	`a = [10,20,30,40,50]`
Size from values	`int a[] = {1,2,3};`	`int[] a = new int[]{1,2,3};`	`a = [1,2,3]`
All zeros	`int a[5] = {0};`	`int[] a = new int[5];` (auto 0)	`a = [0] * 5`
Partial values	`int a[5] = {1,2};` gives 1 2 0 0 0	`new int[]{1,2,0,0,0}`	`a = [1,2] + [0]*3`

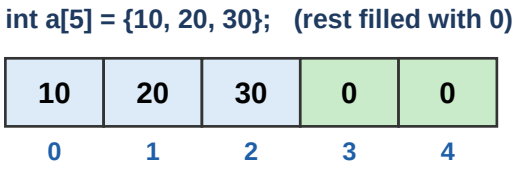

Figure 3: Partial initialization: remaining elements become 0

Initialization using a loop

```
a = [0] * 5
for i in range(5):
    a[i] = i * 10          # a = [0, 10, 20, 30, 40]
```

Initialization from user input

```
n = int(input("How many numbers? "))
a = []
for i in range(n):
    a.append(int(input("Enter value: ")))
```

Types of initialization

1. **Static (compile-time) initialization:** values are written in the program, e.g. a = [10, 20, 30].
2. **Dynamic (run-time) initialization:** values are given while the program runs, using a loop or user input.

Default values (if not initialized)

Language	Default value
Java (numeric array)	0 (boolean: false, object: null)
C (local array)	Garbage (unpredictable) values
C (global / static array)	0

4. Basic Operations on Arrays

Operation	Meaning
Traversal	Visiting each element once, usually with a loop
Access	Reading an element using its index, e.g. a[2]
Update	Changing the value at a given index, e.g. a[2] = 99
Insertion	Adding a new element at a position; later elements shift right
Deletion	Removing an element; later elements shift left to fill the gap
Searching	Finding the position of a value (linear search, binary search)
Sorting	Arranging elements in ascending or descending order
Merging	Combining two arrays into one
Reversing	Changing the order of elements from last to first

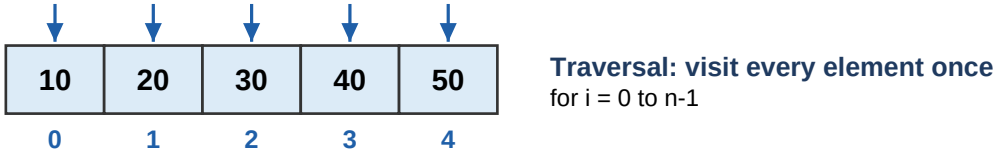

Figure 4: Traversal visits index 0 to n-1 one by one

Insertion in an array

Answer: Insertion places a new value at a chosen index. To create space, all elements from that index onward are shifted one position to the right (starting from the last element).

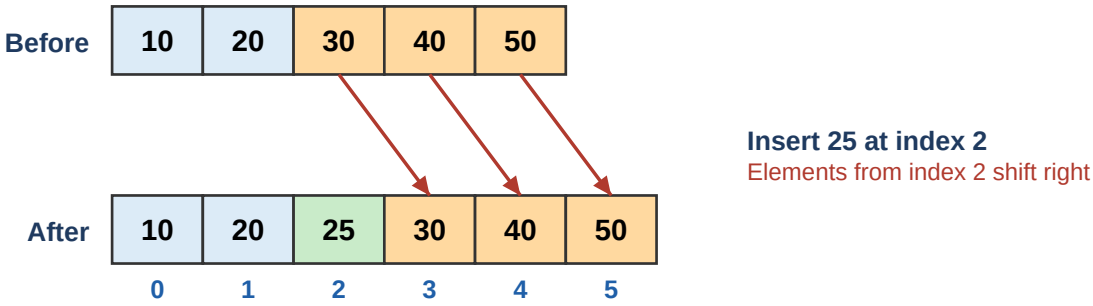

Figure 5: Inserting 25 at index 2 shifts 30, 40, 50 to the right

Deletion in an array

Answer: Deletion removes the element at a given index. All elements after it are shifted one position to the left, and the size reduces by one.

```
Before : 10  20  30  40  50      delete index 1 (value 20)
Shift  : 10  30  40  50  50      (each element moves left)
After  : 10  30  40  50          (length is now 4)
```

Time complexity

Operation	Time
Access / update by index	O(1)
Insert / delete at end	O(1)
Insert / delete at beginning or middle	O(n)
Linear search	O(n)
Binary search (sorted array)	O(log n)

5. Advantages of Arrays

1. **Fast access:** any element is reached directly by its index in O(1) time.
2. **Easy to use:** one name can store many values of the same type.
3. **Easy traversal:** loops can process all elements quickly.
4. **Less code:** avoids creating many separate variables.
5. **Memory efficient:** no extra memory is used for links or pointers.
6. **Cache friendly:** elements are stored together, so access is faster.
7. **Easy sorting and searching:** many algorithms are built for arrays.
8. **Foundation for other data structures:** stacks, queues, heaps, hash tables and matrices use arrays.

6. Disadvantages of Arrays

1. **Fixed size:** a static array cannot grow or shrink after creation.
2. **Costly insertion and deletion:** elements must be shifted, which takes O(n) time.
3. **Memory wastage:** if the array is declared too large, unused space is wasted.
4. **Overflow:** if declared too small, extra data cannot be stored.
5. **Same data type only:** in C and Java, different types cannot be mixed.
6. **Needs continuous memory:** a large array may fail to allocate if free memory is scattered.
7. **No bounds checking in C:** accessing an invalid index may give garbage values or crash the program.
8. **Resizing is expensive:** a bigger array must be created and all elements copied.

7. Types of Arrays

Answer: Arrays are classified mainly by the **number of dimensions**: one-dimensional, two-dimensional and multi-dimensional arrays. They are also classified by size handling (static and dynamic) and by special structure (jagged, sparse, circular, parallel).

A. One-Dimensional (1D) Array

Answer: A one-dimensional array stores elements in a single row (a linear list). Only **one index** is needed to access an element.

```
int a[5] = {5, 10, 15, 20, 25};      a[2]  gives  15
```

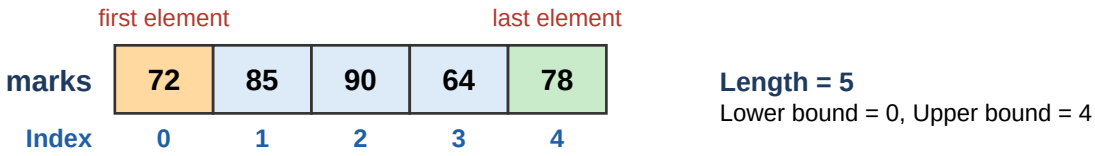

Figure 6: 1D array: one index, e.g. marks[2] = 90

B. Two-Dimensional (2D) Array

Answer: A two-dimensional array stores elements in **rows and columns** like a table or matrix. **Two indexes** (row, column) are needed. Syntax: **datatype name[rows][columns]**.

```
int m[3][3] = { {1,2,3}, {4,5,6}, {7,8,9} };      m[1][2]  gives  6
```

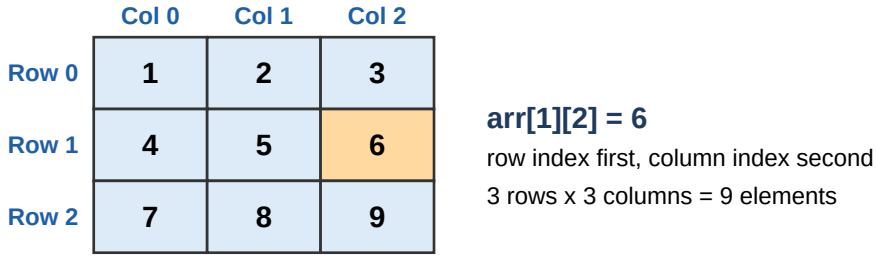

Figure 7: 2D array with 3 rows and 3 columns

8. Differences (Frequently Asked)

1D array vs 2D array

1D array	2D array
Single row of elements	Rows and columns (table / matrix)
One index: a[i]	Two indexes: a[i][j]
Declaration: int a[5]	Declaration: int a[3][4]
Total elements = n	Total elements = rows x columns
Example: list of marks	Example: marks of many students in many subjects

Array vs normal variable

Variable	Array
Stores one value	Stores many values of the same type
Accessed by name only	Accessed by name and index
Needs separate names for many values	One name for the whole group

9. Quick Revision: One-Line Answers

1. Array: a collection of same-type elements stored in contiguous memory under one name.
2. Index: the position number of an element; it starts from 0.
3. Last valid index of an array of size n: n - 1.
4. Address of a[i] = Base address + i x size of element.
5. Access time of an array element: O(1).
6. Insertion or deletion in the middle: O(n) because elements must shift.
7. 1D array uses one index, 2D uses two, 3D uses three.
8. Static array: fixed size. Dynamic array: size can change.
9. Jagged array: rows of different lengths.
10. Sparse array: mostly zeros, so only non-zero values are stored.
11. Circular array: after the last index, go back to index 0 using (i + 1) % n.

Possible exam questions

1. Define an array. Explain its characteristics. (3 to 5 marks)
2. Write the advantages and disadvantages of arrays. (5 marks)
3. Explain declaration and initialization of an array with examples. (5 marks)
4. Explain the types of arrays with neat diagrams. (5 to 10 marks)
5. Differentiate between 1D and 2D arrays; static and dynamic arrays. (3 to 5 marks)
6. Explain insertion and deletion in an array with an example. (5 marks)
7. Find the address of a[5] if the base address is 2000 and each element takes 4 bytes. (2 marks)

Answer to the last question: Address = 2000 + (5 x 4) = **2020**.